

Cualidades del Diseño



Por
Rodrigo Merino
Leonardo Gassman

Versión 1.0
Mayo 2007

Indice

CUALIDADES DE DISEÑO.....	3
ELEMENTOS DE BUEN DISEÑO	3
10 - Mantener una documentación robusta	4
9 - Interface cómoda de usar.	4
8 - Considerar el contexto del proyecto.....	5
7 - Distribuir lógica y asignar responsabilidad de forma correcta	5
6 - Mantener la consistencia y hacerlo intuitivo.....	6
5- Encontrar buenas abstracciones	7
4 - Maximizar la cohesión y minimizar el acoplamiento	8
3 - Eliminar las duplicaciones (Once and only once)	8
2 - Simplificarlo tanto como sea posible.....	8
1 - Hacer que haga lo que tiene que hacer	9

Cualidades de diseño

Antes de comenzar, repasemos algunas definiciones que vamos a usar cuando hablemos de cualidades:

- **Robustez:** Distancia al caos. Mientras menor es la distancia al caos, mayor solidez/robustez posee algo. Un sistema robusto transmite sensación de seguridad tanto al usuario como al programador.
- **Claridad:** Se refiere a la posibilidad de entender el funcionamiento de un sistema, subsistema o una porción de código cualquiera, su objetivo y la forma de solucionar el problema; en particular por gente que no es la que lo construyó. La claridad de un módulo afecta claramente a las posibilidades de modificarlo (flexibilidad).
- **Flexibilidad:** es la capacidad que tiene un sistema para reflejar cambios percibidos en el dominio (sea por mejor comprensión del mismo o porque de verdad cambió) de una manera simple y sencilla. Conceptos relacionados son:
 - **Extensibilidad:** un sistema es extensible cuando pueden incorporarse nuevas características al mismo sin mayor impacto sobre las características actuales.
 - **Mantenibilidad:** un sistema o desarrollo es más mantenible cuanto menor esfuerzo requiere para que el sistema siga funcionando en condiciones distintas a las originales e incluso en las originales.
- **Complejidad:** Es la percepción de muchos elementos relacionados y en los que esas interdependencias manifiestan efectos no esperados que rompen las rutinas y expectativas ordinarias. Es antónimo de simplicidad.
- **Cohesión:** Es la relación entre los elementos de un módulo. Un módulo cohesivo tiene todos sus elementos abocados a resolver el mismo problema.
- **Acoplamiento:** Es el grado de dependencia entre 2 módulos.

Elementos de buen diseño

Muchas veces nos ocurre al tener que leer código que alguien escribió hace algún tiempo que nuestra primera reacción es algo como "Uhh, esto es un asco!!". Pero... ¿en que nos basamos para decir esto? ¿Es sólo una sensación subjetiva o es la aplicación de un criterio objetivo? Dando vuelta la situación... ¿disponemos de las herramientas y conocimientos necesarios para que esa situación no se repita cuando alguien tenga que leer nuestro código o entender nuestro diseño?

Si bien no hay una receta mágica para lograr un diseño de calidad, hay una serie de puntos que de cumplirse correctamente nos pueden acercar bastante a ese objetivo. Los puntos a continuación están ordenados del menor a mayor valor significativo.

TOP 10 LIST



10 - Mantener una documentación robusta

Decimos que la documentación es robusta cuando trata los aspectos más generales y abstractos de un diseño, de manera que su **contenido siga siendo cierto** aun cuando cambien los detalles más específicos y concretos del diseño. En relación con la frase "Distancia al caos", podemos afirmar que tener documentación que no refleja la realidad o desactualizada es una fuente de caos muy importante.

- Código legible es documentación robusta.
- Los JUnit son documentación Robusta



9 - Interface cómoda de usar.

Si al usar algo no es cómodo, por más bueno que sea, no se usa



8 - Considerar el contexto del proyecto

- Tener en cuenta el equipo que va a implementarlo

Al incorporar tecnología y conceptos en nuestro diseño, se debe prestar atención a la experiencia del equipo de programadores del que disponemos para implementar nuestro diseño. Seguramente no se obtendrán buenos resultados si un diseño basado en patrones complejos y tecnologías complicadas debe ser implementado por alguien que se encuentra escribiendo código por primera vez en su vida.

- Considerar el Tiempo disponible de desarrollo
- Costos / beneficios



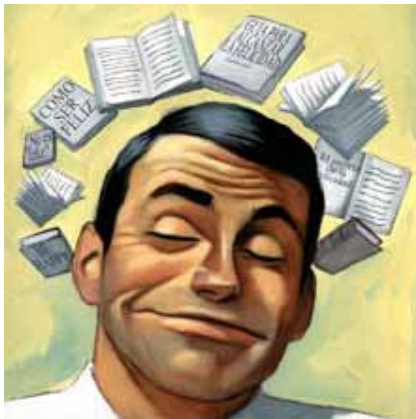
7 - Distribuir lógica y asignar responsabilidad de forma correcta

Esto es básicamente ir en contra del **god-object**. Un síntoma del god-object es cuando nos encontramos que la gran mayoría de los cambios que tenemos que hacer sobre un sistema impactan en parte sobre un mismo módulo y ese módulo se convierte en un cuello de botella de mantenimiento, ya que si tengo un equipo grande haciendo mantenimiento tendré una gran cantidad de gente tocando los mismos archivos incurriendo en el costo grande de **sincronización** que eso conlleva.

No lograr esto, además, atenta contra la **flexibilidad** de un diseño, ya que la cantidad de módulos que deben modificarse ante un cambio de requerimientos es elevado en relación a la cantidad total de módulos de mi sistema.

Básicamente este punto se resume en poder asignar correctamente las responsabilidades (y no caer en los god object o en las responsabilidades excesivamente distribuidas). Debemos encontrar buenas abstracciones para asignar las responsabilidades que les correspondan a cada componente de la aplicación.

Quizás sea bueno citar que la mejor forma de saber cuáles son las responsabilidades de un objeto es concentrarnos en su **interfaz** (su contrato para con el mundo exterior). **Tell don't ask (no hacer get de muchas propiedades para tomar decisiones, delegárselo al objeto).**



6 - Mantener la consistencia y hacerlo intuitivo

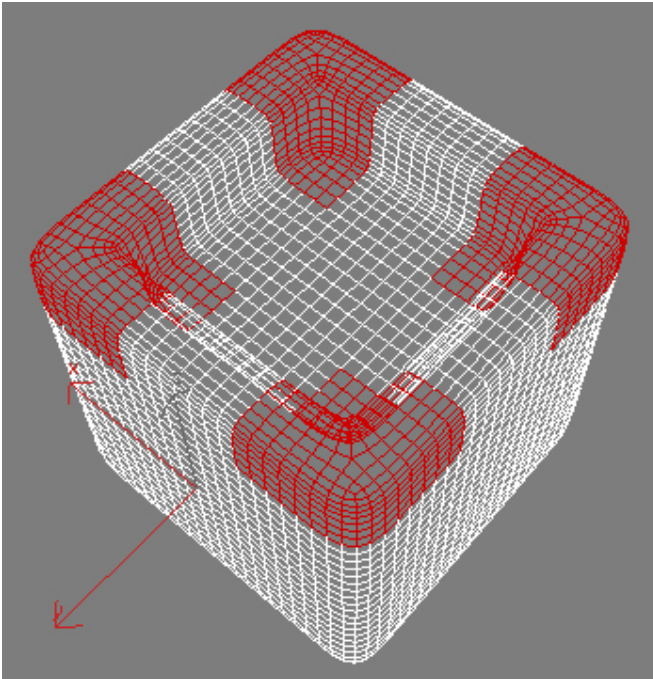
Ser consistente es aplicar los **mismos criterios** uniformemente a lo largo de mi diseño. De esa forma, el diseño se vuelve más **predecible** para un ocasional lector y eso acelera su **comprensión**, es **ágil**.

Por ejemplo, si en alguna parte de mi sistema genero una consulta a la base de datos utilizando un Visitor y en otra lo hago delegándolo en el objeto, no estoy siendo consistente en la forma de acceder a la base de datos. De la misma forma, si a medida que uno lo lee encuentra lo que esperaría encontrar, también se acelera la lectura, ya que no debe detenerse a analizar los motivos que llevaron a una determinada decisión de diseño. Por ejemplo, si estoy analizando un módulo que lee registros de una base de datos y en el medio de ese módulo encuentro algo que genera estadísticas en un archivo, no estoy haciendo un diseño intuitivo.

Ejemplos comunes de falta de consistencia

- Devolver null en lugar de usar Exceptions en búsquedas
- poner a veces get y otras no
- Que cada módulo tenga su propia arquitectura (hibernate – entity beans – llamadas a stored procedures)

La consistencia se ve también en la interface de usuarios, un contraejemplo típico es cuando los botones de aceptar y cancelar no se encuentran siempre en el mismo orden.



5- Encontrar buenas abstracciones

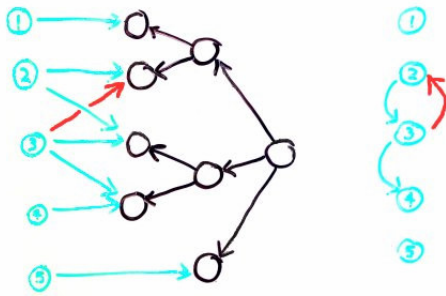
Un sistema no es más que un **modelo**. Como todo modelo, tiene en cuenta ciertas cosas y otras cosas las deja de lado. Encontrar buenas abstracciones es parar nuestro sistema sobre **ideas sustentables**. Las abstracciones principales forman la **metáfora** del sistema. Las buenas abstracciones aportan robustez al sistema. Una porción de código que responde a una abstracción, es lo que permite tener software **reutilizable**.

Utilizar **nombres representativos** me ayuda a encontrar abstracciones. No poder nombrar algo clara y concisamente puede estar dándonos una idea de que no llegamos a una buena abstracción. Igual no es tan taxativo esto ☺, sólo una rule of the thumb.

Esto ayuda enormemente a la autodocumentación del diseño y a la facilidad de lectura del mismo. Los nombres deben ser lo mas específicos y precisos posibles e indicativos de la responsabilidad que tienen asignada.

Otro tema con los nombres, es que pueden utilizarse para explicitar algún contrato (p.ej.: get y set para los beans, test para JUnit, etc), y el no cumplirlos no solo afectará a la estética de mi diseño si no también al correcto funcionamiento de su implementación.

Los nombres son parte del código (inclusive pueden ser parte del contrato/interfaz) y el código es el primer mecanismo de documentación existente. Un código claro se autodocumenta.



4 - Maximizar la cohesión y minimizar el acoplamiento

Otra forma de hablar sobre división de responsabilidades. Fuertemente relacionado con las abstracciones. Componentes más generales tienen mayores posibilidades de ser reutilizados. Tiene que ver con nuestra capacidad de entender el sistema, y con la factibilidad de poder modificarlo.

$$\begin{array}{l}
 1+1 \\
 1+1+1 \\
 1+1+1+1 \\
 \dots
 \end{array}
 \Rightarrow 1 * x$$

3 - Eliminar las duplicaciones (Once and only once)

La duplicación es la némesis por excelencia del buen diseño. Queremos que un mismo problema sea solucionado una sola vez, independientemente de la cantidad de veces que se presente. La duplicación de código aumenta los **costos de mantenimiento** ya que me obliga a repetir el trabajo realizado sobre el elemento duplicado. Muchas veces ocurre que nos encontramos con un problema y lo resolvemos, sin conocer que quizás alguien más en el proyecto ya tuvo ese mismo problema y lo resolvió. Esta es una de las principales causas de aparición de soluciones duplicadas, y puede ser minimizada eficazmente promoviendo una buena comunicación entre los integrantes de un equipo de trabajo. Muchas veces ocurre que se supone que el código que estoy modificando es el único que cumple con tal función cuando no es así, esto lleva a que las modificaciones sobre el código no se vean reflejadas en el comportamiento del sistema, tirando a la basura la robustez del mismo.



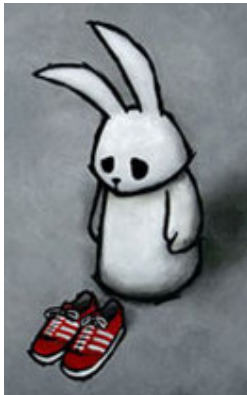
2 - Simplificarlo tanto como sea posible

Debemos distinguir entre **2 tipos de complejidad**: la accidental y la esencial. La complejidad esencial es aquella propia del problema que queremos resolver, mientras que la accidental es aquella complejidad adicional proveniente de la solución. Un buen diseño intenta minimizar la complejidad accidental y maneja la complejidad esencial de manera elegante. Un diseño simple es menos proclive a tener errores y es más mantenible.

La simplicidad o complejidad de un diseño o un código puede utilizarse como criterio para determinar las habilidades de su diseñador: Un diseñador principiante crea soluciones simples para problemas simples, un diseñador avanzado crea soluciones complejas para problemas complejos, un gran diseñador crea soluciones simples para problemas complejos. Si bien el resultado final en el tercer caso es algo simple y quizás minimalista, el proceso para llegar a ese resultado no lo es.

Evaluar los mecanismos de implementación

Hay ciertos mecanismos de implementación que son problemáticos con respecto a la complejidad que se deben tratar de desalentar desde el diseño y utilizar sólo cuando el problema lo amerite. Algunos de estos mecanismos son: reflexión, generación automática de código, aspectos, multi-threading. Esto también se relaciona fuertemente con el primer punto.



1 - Hacer que haga lo que tiene que hacer

Básicamente, el diseño debe cumplir con los requerimientos planteados y tolerar de la mejor forma posible los futuros cambios en esos requerimientos. Se debe tener especial cuidado también en observar que el diseño cumpla con los requerimientos no funcionales, el no hacerlo es un error muy frecuente en el diseño y una de las principales fuentes de frustración en un proyecto.